TeachBack

Explain it to another student — Project Brief

Khutso Mashapu (2588152)
Steven Mabasa (2717404)
Education theme · Skill development
Lecturer: Prof. Hima Vadapalli

Summary. TeachBack is a web app where a university student learns a concept by teaching it to **Kai**, a virtual peer who knows only what the student tells it. Kai adopts the student's explanation *including its errors and omissions* and displays what it has understood as a live concept map, so an incomplete explanation produces a visibly incomplete understanding that the student must repair.

1. The problem

Students often mistake *familiarity* for *understanding*. Re-reading notes makes material feel known, but a student who recognises an idea may still be unable to explain it, connect its parts, answer a "why?" question or correct a misconception. People routinely overestimate how well they can explain things until they are asked to (the *illusion of explanatory depth* [3]). For most students that moment comes in a test or exam, when it is too late to act on.

The tools students use rarely expose this gap. Reading, flashcards and quizzes reward recall and recognition, and give right/wrong verdicts rather than any chance to explain an idea in one's own words. General-purpose AI chatbots make it worse: they explain *to* the student, so the student stays a consumer of explanations. The gap is a **visibility** problem: students cannot see what they don't understand. That makes it an interaction design problem rather than a content problem.

2. Target users

- **Primary users:** university students on conceptual or problem-solving courses, studying on their own, who want to check whether they genuinely understand a topic before they are assessed on it.
- **Prototype scope:** three introductory computer science concepts with well-documented, predictable misconceptions: **recursion** (e.g. believing the base case's value is the final answer [4]), **pointers & references** (believing a pointer holds a copy of the value) and **binary trees** (believing every binary tree is sorted). Documented misconceptions give the virtual peer plausible wrong beliefs to hold.
- **User needs:** a *low-pressure*, private way to practise explaining; follow-up questions that expose gaps; reflection on what they know *without being handed the answer*.
- **Users with particular needs:** less confident students, students working in a language other than their first, and students with visual, motor or reading differences. All of these shaped the design (Section 4).

Grounding. In short sessions, five to six students rate their confidence on several topics from a current course and then explain a high-confidence topic aloud. The difference between their stated confidence and the explanation they give is direct evidence of the gap, and the points where explanations break down inform Kai's questions. TeachBack builds the same check into the product: it asks for confidence before and after each session and compares it with what Kai actually understood.

3. How the design solves it

TeachBack uses the **protégé effect**: students who prepare to teach engage more deeply than students who prepare to be tested [1]. Teachable-agent systems such as Betty's Brain had students teach an agent by building a concept map [2]. TeachBack's contribution is in the interaction design: students teach in **natural language** (typed or spoken), and the interface concentrates on how the learner's gaps are **represented, paced and voiced** back to them.

3.1 Core interaction loop

- **Explain.** The student explains the concept to Kai in their own words. Kai's opening line frames it as a favour between peers ("I've got a test on recursion and I honestly don't get it…"), not a test.
- **Inherit.** Kai adopts exactly what it was told. Each sub-idea on the map becomes *Kai gets this*, *Kai is unsure*, *Kai has this mixed up* or *Not taught yet*. When the explanation is vague, Kai fills the gap with a plausible wrong belief and says it back ("Wait, so when the base case returns 1, that's the final answer?"). The student sees the consequence of their own words.
- **Question.** Kai asks **one** follow-up question at a time about the next unclear or untaught idea, in the voice of a curious peer. It never gives the answer.
- **Inspect & contest.** Tapping any node opens *"Why does Kai think that?"*, which shows Kai's current belief, the student's exact words that produced it, a hint, and a *Challenge this* option.
- **Watch Kai try.** Kai works through a small example (e.g. tracing **factorial(3)**) step by step, using only what it was taught. It visibly fails at the idea the student explained wrongly or left out, which turns an abstract gap into a concrete error the

student can repair.

- **Reflect.** A reflection summary lists the ideas explained clearly, the misconceptions repaired and the ideas worth revisiting, compares confidence before and after with Kai's understanding, and suggests next steps.

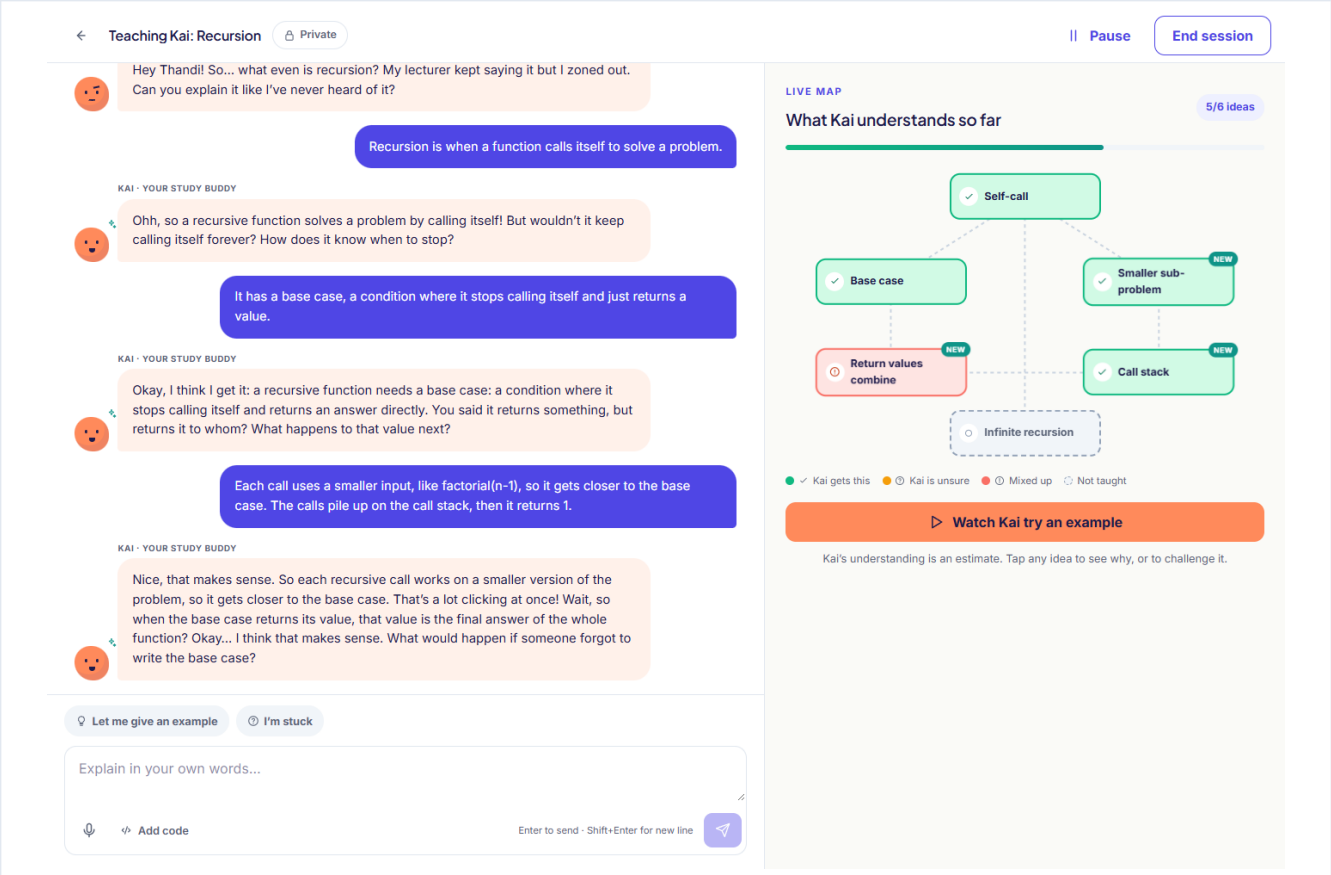


Figure 1. The teaching session in the working prototype. Left: the conversation with Kai. Right: Kai's live understanding map. The student has explained four ideas clearly (green), but the phrase "then it returns 1" led Kai to believe the base case's value is the final answer (coral). Watch Kai try is now unlocked and will expose that belief.

3.2 Prototype

The design moved from problem framing to a Figma prototype and then to a **working web app** (React + TypeScript) that implements all three topics end to end. Kai's understanding model runs entirely in the browser: each sentence of an explanation is matched against bounded, hand-authored rules for every idea in the topic (clear, explicitly wrong, or vague), and Kai's reply and question are generated from the resulting map. Students can create a profile, pause and resume sessions, replay past conversations, track progress across topics and adjust accessibility settings. All of this works without a database or server.

4. Design rationale

The central design risk is that teaching collapses into assessment. If Kai reads as an examiner, the interaction becomes a quiz. Every decision below aims to keep the experience peer-like, transparent and under the student's control.

Concern	How the design addresses it
Usability	Gaps are **visible at a glance** on the map rather than buried in dialogue, with node changes highlighted as "NEW". Kai asks one question at a time, with a choice of pace (Relaxed / Normal / Curious). An *I'm stuck* panel offers nudges (an analogy, tracing a small input, or the hint for the next weak idea) instead of answers. Sessions save automatically and can be paused and resumed. On phones, the chat and the map are separate tabs. Pasting long text triggers a gentle reminder that Kai learns best from the student's own words.
Safety & misleading feedback	The biggest risk is a virtual peer that models a concept wrongly and *reinforces* a misconception. In response: (1) content is scoped to bounded CS concepts where correctness can be defined; (2) every judgement is labelled as *an estimate, not a grade*; (3) Kai's beliefs are always traceable to the student's own quoted words; (4) the student can **challenge** any node or reopen a finished session, and disputed ideas are marked; (5) Kai never states the correct answer, so the system cannot present a wrong "model answer" as truth.
Psychological safety	No scores, grades, timers or leaderboards. Gaps are framed positively ("Finding a gap isn't failure. It's the useful part.") and repaired misconceptions are celebrated on the Progress page. The student can pause at any time ("Kai will wait right here").

Concern	How the design addresses it
Privacy	Records of what a student *could not* explain are sensitive, so they stay under the student's control. **No database and no server:** profiles and sessions are stored only in the student's own browser, and Kai's model runs on the device, so explanations are not uploaded. Profiles are protected with a PIN, stored only as a salted PBKDF2 hash. To move devices, students download a backup file **encrypted with their PIN** (AES-GCM). Students can turn off transcript storage, delete individual sessions (with undo), delete all sessions, or delete their profile. Nothing is ever shared with lecturers. One trade-off: browser voice input may send audio to the browser vendor's speech service, so typing is always available and the transcript is shown for review.
Bias & fairness	Kai judges *ideas*, not grammar, spelling or accent. A *simpler words* mode (offered at sign-up for students working in a second language) shortens Kai's replies. The rule-based model is deliberately transparent: every judgement shows its evidence and can be overridden. We acknowledge that unusual but correct phrasing can be missed, and contestability is the safeguard against that.
Accessibility	Status is never conveyed by colour alone: every state combines colour, icon, text label and border style (dashed for untaught). Type or speak input; adjustable text size; a readable font option (Atkinson Hyperlegible); high-contrast, dark and reduced-motion modes; full keyboard navigation with visible focus; focus moved to each page heading for screen readers; ARIA live regions for Kai's replies; and touch targets of at least 44 px. There is no time pressure anywhere.

4.1 Key design decisions and alternatives considered

Decision	Alternative rejected, and why
Kai is a confused peer, not a tutor	A tutor chatbot that explains and grades. It would return the student to consuming explanations and make the interaction feel like an exam.
Teach in natural language	Building a concept map by hand (as in Betty's Brain [2]). Free explanation is closer to how students actually explain to friends, and it surfaces vague wording, which is where the gaps hide.
Understanding shown as a map beside the chat	Feedback only inside the dialogue. Gaps would be buried in text. The map makes them visible at a glance and gives a stable picture of progress.
Kai inherits errors instead of flagging them	Marking answers right or wrong. Seeing Kai believe (and then fail with) your own mistake is more memorable and less judgemental than a red cross.
Local-first storage with encrypted backups	A cloud database with accounts. It would add cost and a privacy risk for sensitive records of failure. A PIN-encrypted file still lets students move between devices.

5. Success criteria and evaluation

The design succeeds if the interaction (i) gets students explaining in their own words, (ii) exposes a meaningful knowledge gap the student had not previously noticed, (iii) has Kai's questions experienced as helpful rather than obstructive, and (iv) produces a reflection summary that students consider actionable. These are evaluated through **think-aloud sessions** with students from the target group, alongside the built-in comparison between confidence ratings and Kai's final understanding.

Limitations and next steps. Kai's rule-based understanding is bounded to three topics and to the phrasings its rules anticipate. Future work includes broader topic coverage, a hybrid language model constrained by the same visible concept map, and a study of whether gaps found with TeachBack translate into better assessment outcomes.

References

[1] Chase, C. C., Chin, D. B., Oppezzo, M. A., & Schwartz, D. L. (2009). Teachable agents and the protégé effect: Increasing the effort towards learning. *Journal of Science Education and Technology, 18*(4), 334–352.

[2] Biswas, G., Leelawong, K., Schwartz, D., Vye, N., & TAG-V. (2005). Learning by teaching: A new agent paradigm for educational software. *Applied Artificial Intelligence, 19*(3–4), 363–392.

[3] Rozenblit, L., & Keil, F. (2002). The misunderstood limits of folk science: An illusion of explanatory depth. *Cognitive Science, 26*(5), 521–562.

[4] Kahney, H. (1983). What do novice programmers know about recursion? *Proceedings of CHI '83*, 235–239. ACM.